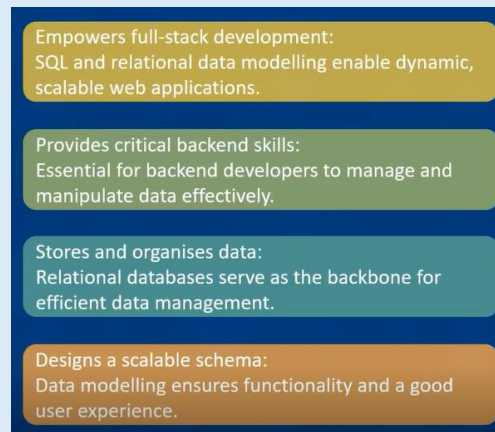


Module 9: Relational Databases and SQL

Module Overview

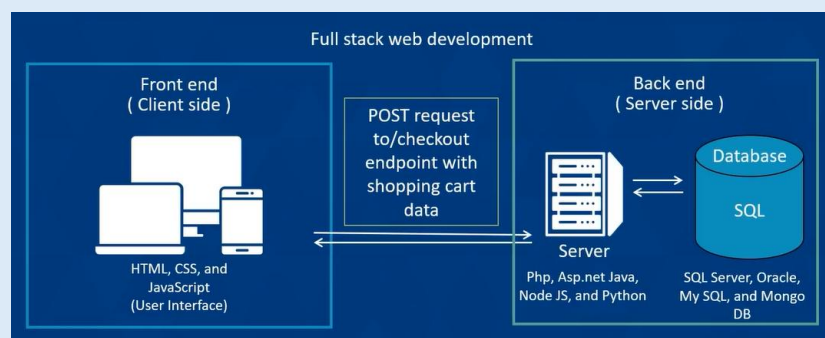
- Relational databases and SQL are essential tools for full stack and backend developers building scalable, data-driven web applications.
- Data modelling is crucial for designing a database schema that accurately represents application data and ensures usability.



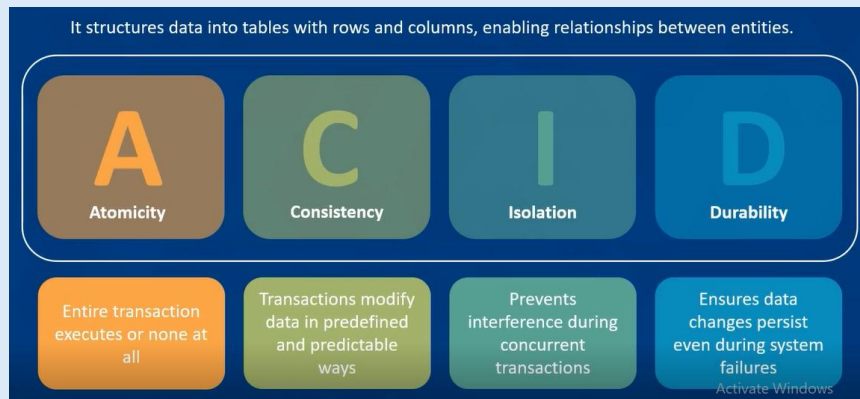
- SQL enables reliable interaction with databases by allowing structured retrieval, insertion, updating, and deletion of data.
- Database relationships and relational models form the backbone of organised data storage and efficient data retrieval in applications.
- The module covers key topics including relational modelling, entity-relationship diagrams, and writing both basic and advanced SQL queries.

Introduction to Database Systems

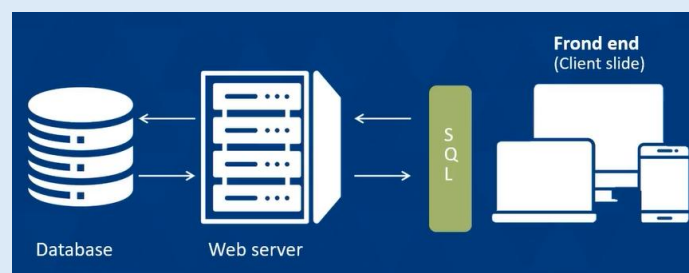
- Databases in backend development serve as structured repositories for storing, retrieving, and modifying application data efficiently.
- Backend systems often receive data from the frontend through HTTP requests and manage it using RESTful APIs or ORM libraries.



- Relational databases like MySQL and PostgreSQL structure data into tables and ensure reliability through ACID-compliant transactions.



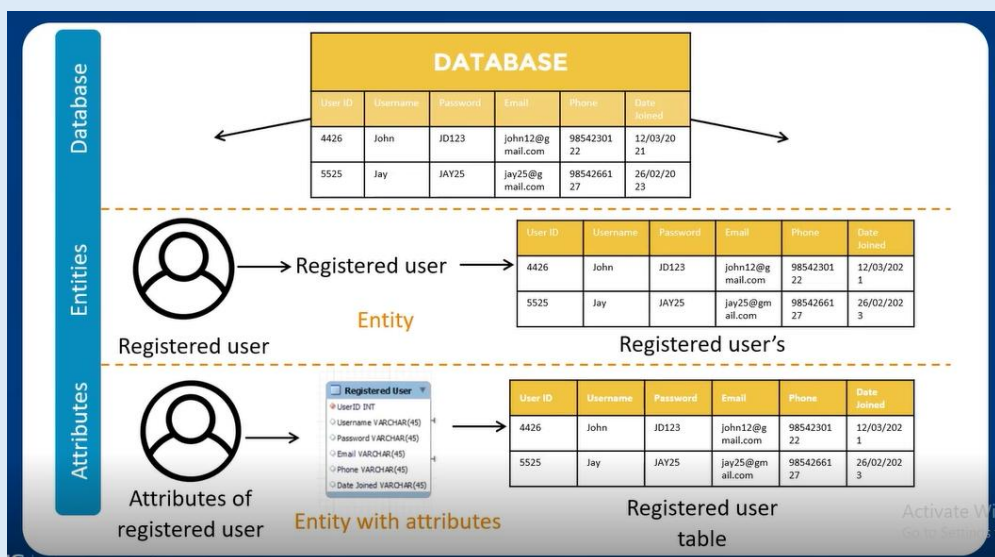
- SQL is the standard language for interacting with relational databases, though dialects may vary depending on the database system.



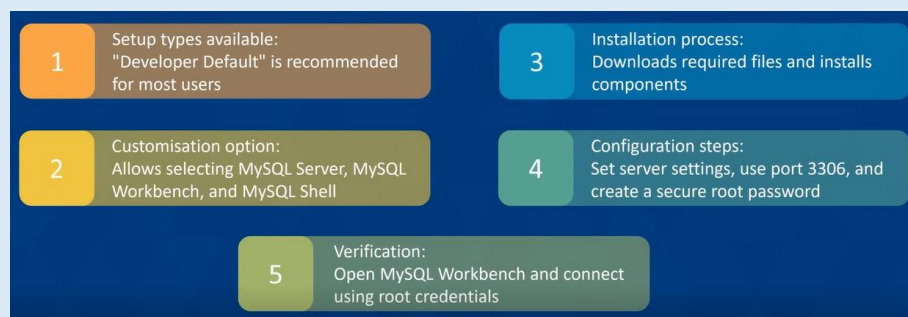
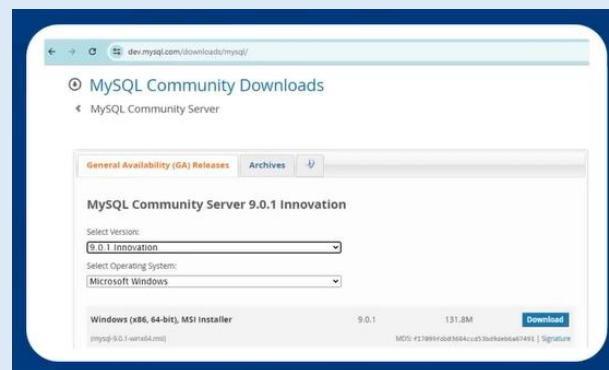
- Enterprise database options include open-source solutions and cloud-based platforms, with selection based on scalability, support, and technical needs.

Database Modelling and Data Model

- Semantic or conceptual modelling provides a high-level view of a business domain using terms and concepts familiar to business users.
- A data model organises data elements, their relationships, and attributes to reflect real-world entities like users, addresses, and payment methods.
- Relationships between entities define valid associations and constraints, such as one address and one payment method per registered user.

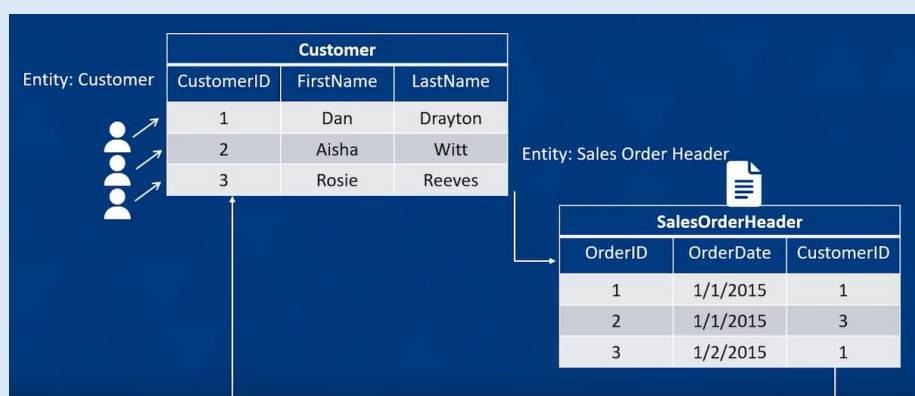


- Business rules clarify data requirements and enforce consistency, guiding developers in structuring accurate and implementation-ready data models.
- MySQL installation involves downloading the installer, selecting key components like Workbench and Shell, and configuring the server with secure settings.

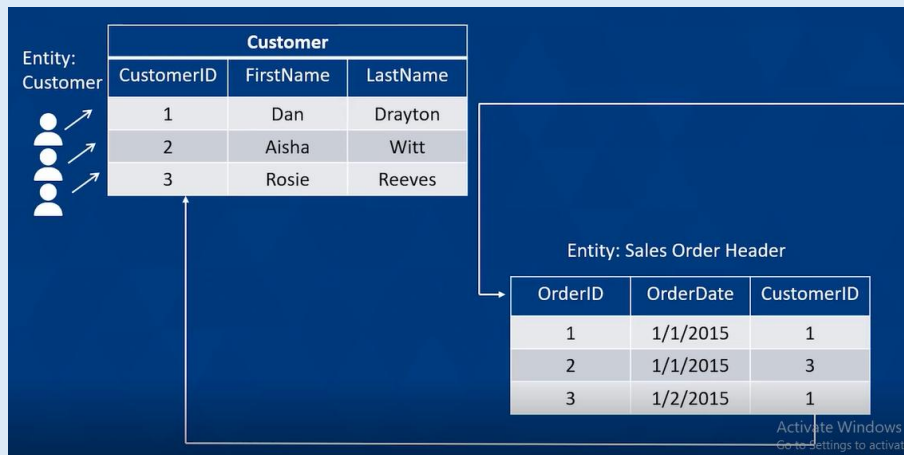


Relational Database Model

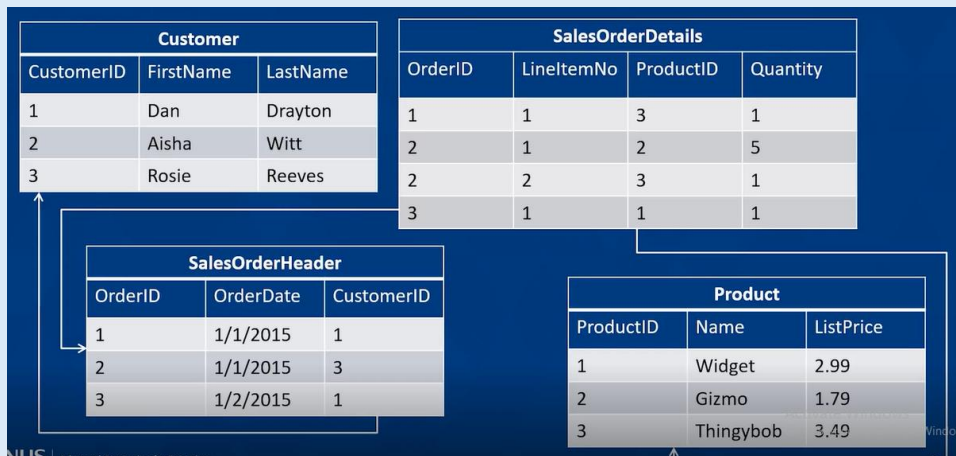
- Schemas within a database help group related tables logically, allowing better organisation and clarity in complex data environments.
- Tables in a relational model map to nouns like 'Customer' or 'Order', with rows as records and columns as descriptive attributes.
- Relationships allow efficient data linkage between entities, avoiding data duplication and supporting scalability in data design.



- Key constraints include primary keys for uniqueness, alternate keys as backup identifiers, and foreign keys to establish valid cross-table references.

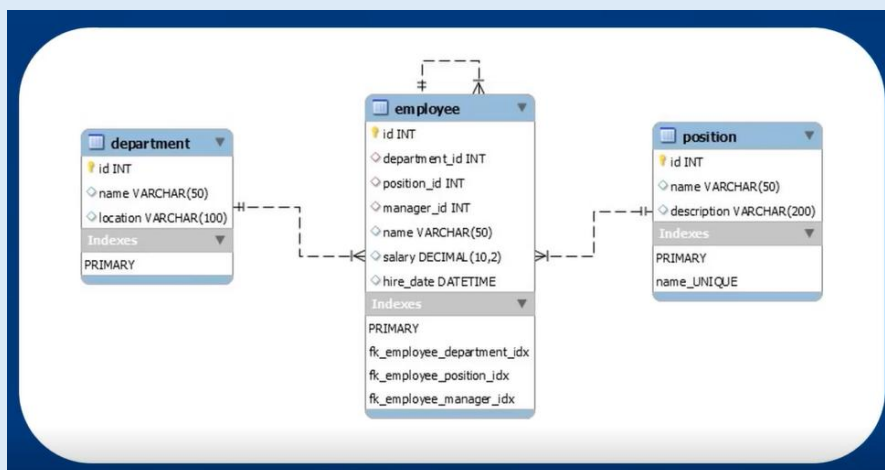


- Enforcing referential integrity prevents invalid or orphaned relationships, ensuring data consistency across connected database tables.



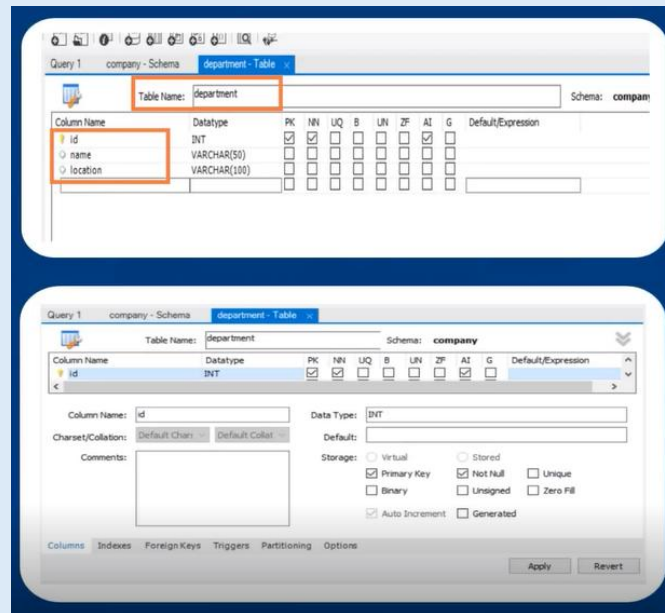
Build your database and create an ERD

- The Entity Relationship Diagram provides a visual representation of the database structure, helping identify and resolve design issues early.

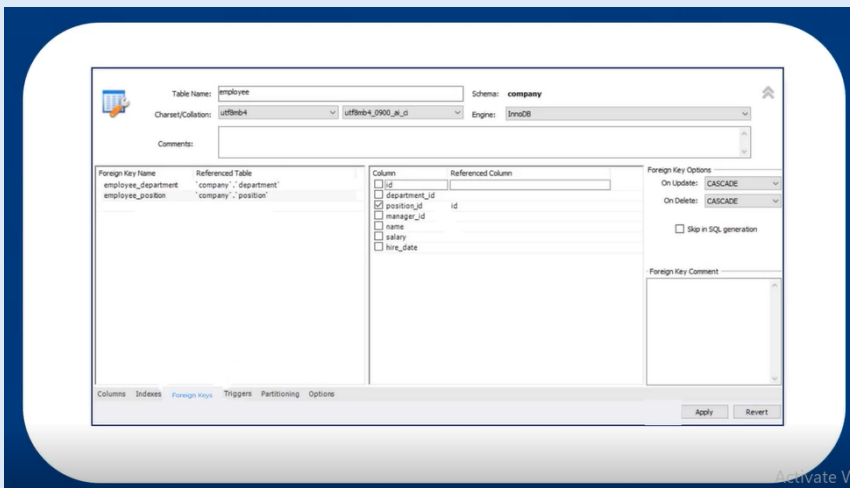
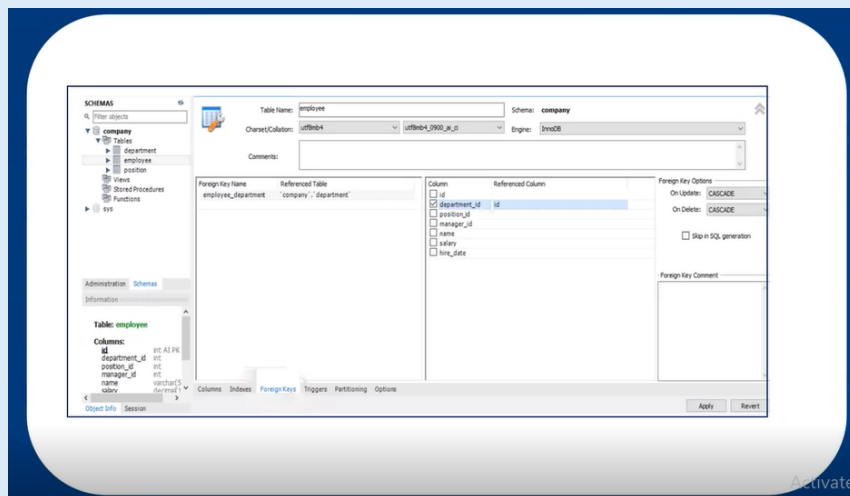


- MySQL Workbench supports model-driven design with features like graphical modelling, reverse engineering, and forward engineering to scripts or databases.

- Creating tables such as department, employee, and position involves defining primary keys, attributes, and appropriate data types within the MySQL Workbench.



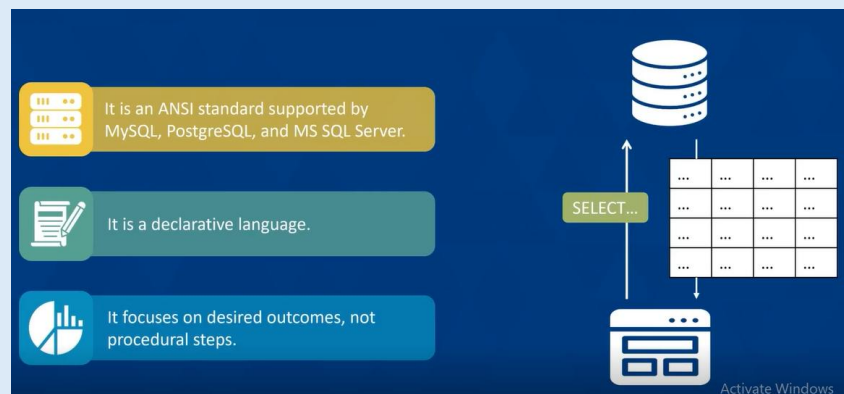
- Foreign keys establish relationships across tables, with cascading options for updates and deletions to maintain referential integrity throughout the database.



- The reverse engineering process in MySQL Workbench generates the ER Diagram from existing tables, offering a clear conceptual view for backend development.

Write SQL Data Manipulation Language

- Data Manipulation Language (DML) in SQL includes key commands like SELECT, INSERT, UPDATE, and DELETE for interacting with relational databases.
- SQL operates declaratively, allowing developers to state desired outcomes rather than control flow, unlike procedural languages such as Python or Java.



- SELECT statements retrieve data, with variations such as SELECT * for all columns or specific columns with calculated fields and aliases for readability.

SELECT statement

- It fetches data from a database using a specific syntax.
- To retrieve all columns: SELECT * FROM table_name;

```
SELECT column1, column2, column_n...
FROM your table;
```

- INSERT and UPDATE statements modify table data by specifying column-value pairs, with optional use of CAST to convert data types explicitly.

INSERT statement

- It adds new records to a table.
- Each value corresponds to a specified column.

```
INSERT INTO Table (column_1, column_2, column_n)
VALUES (value1, value2, value_n);
```

UPDATE statement

- It modifies existing data in a table.
- The SET keyword assigns new values, and WHERE filters the rows to update.

```
UPDATE Table
SET column_1 = value1, column_n = value_n,
[ WHERE condition];
```

- DELETE statements remove specific records using WHERE clauses, ensuring accurate targeting of rows without affecting unintended entries.

DELETE statement

- It removes rows from a table.
- The WHERE clause specifies which rows to delete.

```
DELETE FROM Table
[ WHERE condition];
```

Write Advanced SELECT Queries

- The WHERE clause filters rows based on specified conditions, using comparison operators (=, !=, >=, <=, etc.) and logical operators like AND and BETWEEN.

Contains one or more conditions that evaluate to TRUE or FALSE

Returns rows only when the condition is TRUE

=	Equals
<>	Not equals
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to

- Regular expressions (e.g., ^Cam.*n\$) can be used within WHERE clauses to match complex text patterns using wildcards such as *, _, [], and [^].

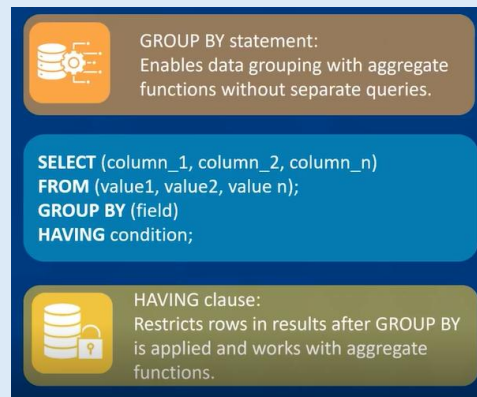
Regular expressions define patterns for complex searches in the WHERE clause.

The search pattern starts with ^ (hat) and ends with \$ (dollar sign).

```
WHERE firstname REGEXP '^Ca[m].*n$'
```

.	Matches strings of zero to any characters
_	Matches any single character
[]	Any single character within the specified range (e.g., [a-f]) or set (e.g., [abcdef])
[^]	Any single character not within the specified range (e.g., [^a-f]) or set (e.g., [^abcdef])

- The GROUP BY clause groups records by one or more columns, allowing aggregate functions like COUNT(), AVG(), or SUM() to be applied on grouped data.

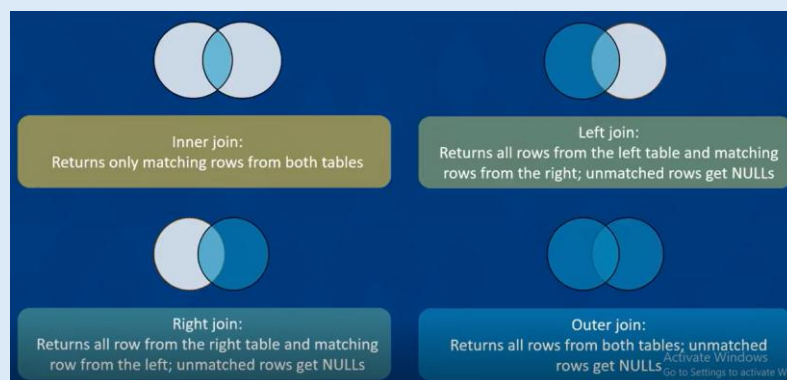


- HAVING clause filters grouped results after aggregation, such as filtering only groups where the average list price exceeds \$700.
- SQL query execution order differs from written syntax: the engine processes FROM, WHERE, GROUP BY, HAVING, and then SELECT, which helps optimise and troubleshoot queries.



Write SQL Joins to combine data from multiple tables

- Join operations allow retrieval of data from multiple tables using a SELECT query, provided that a common or related column exists between the tables.
- SQL supports various types of joins—inner, left, right, and outer—each determining how matched and unmatched data is included in the result.



- In an inner join, only rows with matching values in both tables are returned, whereas a left join includes all rows from the left table and matched rows from the right.
- Joins can involve more than two tables, such as combining sales order headers, details, and product information using appropriate join keys and the ON clause.
- SQL Server uses a logical Cartesian join internally, filtered by the ON clause to match specific fields, with the final output returning only the selected columns.

```

SELECT
    oh.OrderDate,
    oh.SalesOrderNumber,
    p.Name AS ProductName,
    od.OrderQty,
    od.UnitPrice,
    od.LineTotal
FROM SalesLT.SalesOrderHeader AS oh
JOIN SalesLT.SalesOrderDetail AS od
    ON od.SalesOrderID = oh.SalesOrderID
JOIN SalesLT.Product AS p
    ON od.ProductID = p.ProductID
WHERE OrderDate BETWEEN '2008-06-01' AND '2008-06-30'
ORDER BY oh.OrderDate, oh.SalesOrderID, od.SalesOrderDetailID;

```

OrderDate	SalesOrderNumber	ProductName	OrderQty	UnitPrice	LineTotal
2008-06-01 00:00:00	SO71774	ML Road Frame-W - Yellow, 48	1	356.8980	0.0000
2008-06-01 00:00:00	SO71774	ML Road Frame-W - Yellow, 38	1	356.8980	0.0000
2008-06-01 00:00:00	SO71776	Rear Brakes	1	63.9000	0.0000
2008-06-01 00:00:00	SO71780	ML Mountain Frame-W - Silver, 42	4	218.4540	0.0000
2008-06-01 00:00:00	SO71780	Mountain-500 Silver, 46	2	461.6940	0.0000
2008-06-01 00:00:00	SO71780	Mountain-500 Silver, 52	6	112.9980	0.0000
2008-06-01 00:00:00	SO71780	HL Mountain Frame - Silver, 38	2	818.7000	0.0000
2008-06-01 00:00:00	SO71780	Mountain-500 Black, 42	1	323.9940	0.0000
2008-06-01 00:00:00	SO71780	LL Mountain Frame - Black, 48	1	149.8740	0.0000
2008-06-01 00:00:00	SO71780	HL Mountain Frame - Black, 42	1	809.7600	0.0000

Module Summary

- Organise and manage structured data using relational databases through conceptual, logical, and physical data models.
- Enforce entity and referential integrity by defining entities, relationships, and applying primary and foreign keys.
- Manipulate data dynamically using SQL commands such as INSERT, UPDATE, and DELETE.
- Construct advanced SELECT queries using aggregate functions and clauses like WHERE, GROUP BY, and HAVING for in-depth analysis.
- Retrieve related data across multiple tables using JOIN operations, including INNER, LEFT, RIGHT, and SELF joins.



Managing Data with SQL:
Insert, update, and delete data using SQL commands for dynamic data management.



Advanced SELECT Queries:
Use aggregate functions, WHERE, GROUP BY, and HAVING clauses for complex data retrieval.



SQL Joins:
Combine data from multiple tables using INNER, OUTER, and SELF joins to fetch precise information.

THE END